

FinRelief AI — Development Guide

SmartBridge Internship Program

Version 2.0.0

1. Prerequisites

- Python 3.11 or later
- Node.js (for the React + Vite frontend)
- A Neon PostgreSQL connection string (for the production-equivalent database)
- A Google Gemini API key (optional — the app runs without one, using the fallback letter template)

2. Project Structure

The repository is a monorepo split into two independently runnable services:

```

Fin-track-prototype/
├── backend/
│   ├── main.py           # FastAPI entry point, wires up all routers
│   ├── auth.py           # Registration, login, JWT handling
│   ├── database.py       # SQLAlchemy engine + session setup
│   ├── models.py        # Database models
│   ├── schemas.py       # Pydantic request/response schemas
│   ├── limiter.py        # SlowAPI rate limiting config
│   └── routers/
│       ├── loans.py
│       ├── settlement.py
│       ├── letters.py
│       └── dashboard.py
│   ├── utils/
│       ├── calculations.py # Debt stress / DTI / settlement logic
│       ├── gemini_client.py # Gemini call + fallback template
│       └── pdf_generator.py # ReportLab PDF export
│   └── tests/            # Pytest suite (SQLite-backed)
├── frontend/
│   ├── src/
│       ├── api/client.js  # Axios instance, auth header handling
│       ├── context/       # Auth state
│       ├── features/      # auth, dashboard, loans, settlement, letters
│       └── components/
│   ├── vite.config.js
│   └── render.yaml       # Backend deployment config for Render

```

The frontend's features/ folders map one-to-one with the backend's routers/ — each feature area (loans, settlement, letters, dashboard) has a matching folder on both sides.

3. Environment Variables

Both backend/.env.example and frontend/.env.example document the required variables. Actual values belong only in a local .env file, and should never be committed.

Variable	Where	Required	Purpose
DATABASE_URL	backend	Yes	Full PostgreSQL connection string (Neon). The app will not start without it.
FINRELIEF_SECRET_KEY	backend	Yes	Signs JWT tokens. Should be a long, random string, unique per environment.
GEMINI_API_KEY	backend	No	Enables AI letter generation. If missing or invalid, the app falls back to the static template automatically.
FRONTEND_URL	backend	Yes	Used for CORS configuration, so only the deployed frontend origin can call the API.
VITE_API_URL	frontend	Yes	Base URL the frontend uses to reach the backend API.

4. Running the Backend Locally

```
cd backend
pip install -r requirements.txt
uvicorn main:app --reload
```

Once running, interactive API documentation is available at <http://localhost:8000/docs>.

5. Running the Frontend Locally

```
cd frontend
npm install
npm run dev
```

The Vite dev server will print the local URL it's running on, typically `http://localhost:5173`.

6. Running the Test Suite

The backend's automated tests run against an isolated SQLite database, kept completely separate from the Neon production database, so tests never touch real data.

```
cd backend
pytest
```

The suite covers financial calculation logic, authentication flows, and the Gemini fallback path (forcing Gemini to fail to confirm the static template is returned correctly).

7. Code Style

- Backend Python code follows PEP 8.
- Frontend JavaScript is linted with `oxlint` and `ESLint`.
- Commit messages follow the Conventional Commits format (e.g. `feat: add settlement snapshot endpoint`, `fix: correct DTI rounding`).

8. Deployment

Component	Platform	Notes
Frontend	Vercel	Serves the Vite production build
Backend	Render	Runs <code>uvicorn main:app</code> ; environment variables are configured in the Render dashboard, not committed to the repo
Database	Neon	Already cloud-hosted; no separate migration step is needed for deployment

Because Render and Vercel wipe local disk storage on every redeploy, all persistent data must live in Neon rather than any file-based store — this is also the reason SQLite is used only for tests, never for production.

9. Recommended Local Development Order

1. Start the backend first and confirm `/health` reports the database as connected.
2. Confirm `/docs` loads and manually exercise the auth endpoints via Swagger.

3. Start the frontend and confirm it can register/log in against the local backend.
4. Use the /demo/seed endpoint (or the one-click demo login) to populate sample loans without entering real financial data, and exercise the settlement and letter-generation flows end to end.